Apache Paimon

A practical primer — how the format works internally, and how it differs from Delta Lake

Internal learning document

A from-the-ground-up explanation of Paimon's internal design: the LSM-tree storage model, merge-on-read, compaction and the read/write table modes, sequence-based ordering, the four merge engines, and the changelog/CDC model — with the Delta Lake contrast drawn out at every step. Prepared as a shareable team reference, based on the Apache Paimon documentation.

Contents

HOW TO READ THIS

Each section builds on the previous one. The single load-bearing idea is in Sections 3–4 (copy-on-write vs merge-on-read, and the LSM tree). Once that clicks, everything later — compaction, merge engines, the changelog model — follows directly from it.

SECTION 1

The three layers, and what a table format provides

People blur three different things together, and that is the root of most confusion. Keep them separate.

1. **Storage** — where the bytes physically live: S3, ADLS, GCS. A dumb bucket of files with no notion of "tables."
2. **File format** — how a single file is laid out internally: Parquet, ORC, Avro. Parquet is *columnar*, storing each column's values together, so reading one column (e.g. summing a revenue column) touches far less data than a row store.
3. **Table format** — the bookkeeping layer that makes a folder of files behave like one coherent table. Delta, Iceberg, Hudi, and **Paimon** all live at this layer.

KEY IDEA

Delta and Paimon are both *table formats* sitting on top of Parquet files in object storage. So a Delta-vs-Paimon comparison is almost never about the storage or the file format — those are usually identical. It is about **the bookkeeping that decides what counts as the current state of the table.**

What a table format gives you that raw files can't

Object storage has no transactions. If you simply call a folder of 100 Parquet files "a table," you immediately hit:

- A reader arriving mid-write sees half-written, inconsistent files.
- There is no safe way to update or delete rows — and no agreement on what a reader sees during a rewrite.
- Two writers running at once can silently clobber each other.
- There is no way to ask "what did this table look like an hour ago?"

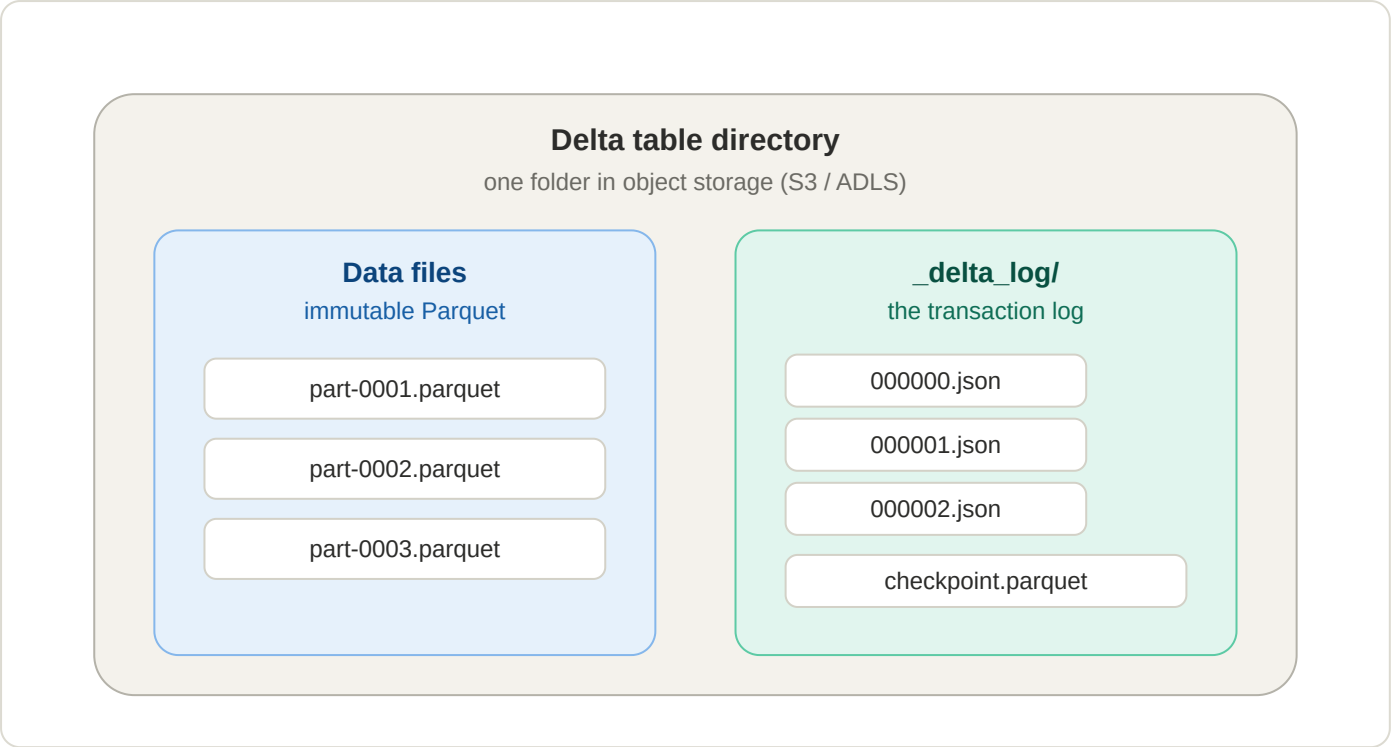
A table format fixes all of this with one move: **readers never look at the folder directly.** They read a metadata/log layer that tells them exactly which files are currently valid. From that single indirection you get the four things every modern table format advertises: **ACID transactions, snapshot isolation, time travel** (read an older version), and **schema evolution**. Delta and Paimon both provide these; they differ in *how* the metadata is structured and, crucially, in how updates are physically applied.

SECTION 2

Delta Lake from scratch (the baseline)

Paimon is easiest to understand as "Delta, with one big idea swapped out." So first, precisely how Delta works.

A Delta table is just a directory containing two kinds of things: immutable **Parquet data files**, and a `_delta_log/` folder of numbered JSON **commits**. A commit holds no data — only a small list of *actions* such as "add file X (with these stats)" and "remove file Y." The commit numbers increase strictly: `000000.json`, `000001.json`, and so on.



A Delta table = immutable data files + an append-only transaction log that decides which files are live.

How a reader reconstructs the table

The current state of a Delta table is *derived*, not stored directly: replay every commit in order, tracking which files have been "added" but not yet "removed." The surviving set of Parquet files **is** the table right now. The folder might physically hold 50 files while the log says only 12 are live — readers trust the log, never a folder listing. From this you get:

- **Atomicity** — a write only counts once its JSON commit lands; half-written data files are invisible because no commit references them.
- **Time travel / snapshots** — replay the log only up to commit N to see an older version; the old data files are still present, frozen.
- **Optimistic concurrency** — two writers attempt the next commit number; one wins, the other detects the conflict and retries.

Replaying thousands of commits would get slow, so Delta periodically writes a **checkpoint** (a Parquet snapshot of "here is the full set of live files as of commit N"). Readers start from the latest checkpoint and replay only the few commits after it. (Paimon has an analogous idea — snapshots pointing at manifest lists — so the pattern carries over.)

The defining trait: copy-on-write

Because Parquet files are immutable, Delta cannot edit a single row in place. To change *one* row, it must:

1. Find the file that contains the row (using the per-file min/max stats in the log).
2. Read that entire file.
3. Write a brand-new file with the one row changed and everything else copied across.
4. Commit: "remove the old file, add the new file."

This is **copy-on-write (COW)**. Reads are trivial and fast — just open the live files, no merging needed — but any write that touches existing rows rewrites whole files.

THE VILLAIN: WRITE AMPLIFICATION

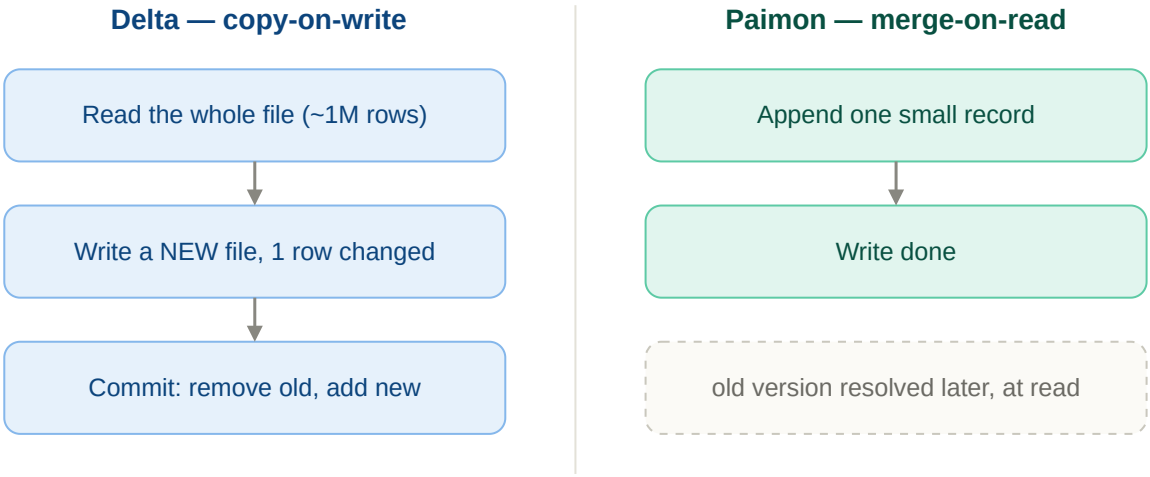
Now imagine a stream delivering thousands of single-row updates per second. Copy-on-write would have Delta rewriting large files continuously — a *tiny logical change* causing a *huge physical write*, over and over. That ratio, write amplification, is the exact pain Paimon was built to remove.

SECTION 3 — THE CORE

The fundamental trade-off: copy-on-write vs merge-on-read

Everything distinctive about Paimon comes from one swap in *when you pay the cost.*

Delta's instinct is to do the work at **write** time so reads stay simple (copy-on-write). Paimon's instinct is the opposite: never rewrite — append the change as a tiny new file, and resolve "which version wins" later, at **read** time. That is **merge-on-read (MOR)**, and it is Paimon's default.



Same logical change. Delta rewrites a large file; Paimon writes about one row.

Copy-on-write pays a large cost on every update; merge-on-read pushes a small amount of work to read time instead.

When order #555 changes to `shipped`, Paimon writes one tiny record ("PK 555 → shipped, sequence 9001") into a small new file. The old `pending` version still physically exists; at read time Paimon keeps the newest. Write amplification essentially vanishes. The cost didn't disappear, though — it moved to the read path (you now have to merge versions) and to background compaction (covered in Sections 6–7). The genius is that this moved cost is *cheap and amortized*, whereas Delta's was *expensive and synchronous on every write.*

THE MIRROR IMAGE

Delta pays at write so reads are trivial. Paimon pays a little at read so writes are trivial. Neither is "better" in the abstract — they're tuned for opposite workloads. Delta suits batch-heavy, append-mostly analytics; Paimon suits high-frequency streaming updates by primary key.

SECTION 4 — THE CORE

Inside Paimon: buckets, the LSM tree, sorted runs & levels

If every update just drops a tiny new file, what stops reads from drowning in millions of files? An LSM tree.

Merge-on-read only works because Paimon *organises* those small files using a **log-structured merge-tree (LSM tree)**, the same structure databases like RocksDB and Cassandra use. Two structural facts come first.

Buckets — the unit of parallelism

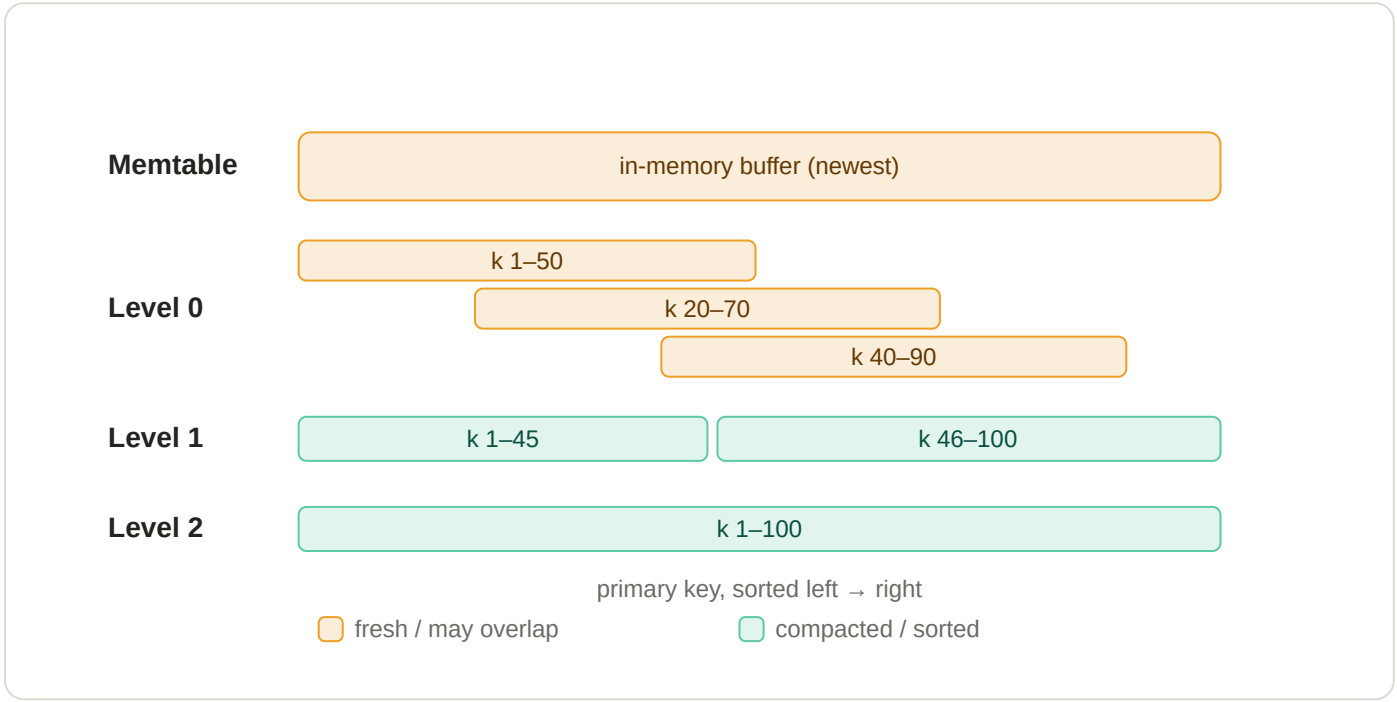
A table (or each partition of it) is divided into a fixed number of **buckets** by hashing the primary key (or a chosen bucket key). Each bucket is an **independent LSM tree**. Because a given key is hashed to a fixed bucket, every version of that key lives together in one bucket. The bucket count caps write/read parallelism: a bucket is the smallest unit of work. Too few buckets throttles throughput; too many creates lots of small files.

The memtable — where writes land first

New writes first go into an in-memory buffer (the **memtable**), sorted by primary key. When the writing engine checkpoints, the memtable is flushed to disk as a small sorted file. Notably, Paimon keeps **no write-ahead log** — it relies on the compute engine's checkpoint mechanism to recover any in-memory data after a failure. (This is the one place the engine, e.g. Flink, is structurally relevant.)

Levels and sorted runs

Within a bucket, data files are arranged into numbered **levels**, organised as **sorted runs**. A sorted run is a set of files where the key ranges do not overlap; within each file, rows are sorted by primary key.



One bucket's LSM tree. Fresh files (Level 0) can overlap on the key axis; compacted files (Level 1+) are sorted and non-overlapping.

Read the diagram top-to-bottom as "newest & messy → oldest & tidy":

- **Level 0** holds freshly flushed files. Their key ranges can overlap — key #45 might appear in several of them (Section 5 explains why).
- **Level 1 and below** are produced by compaction. Within each level the runs are sorted and their key ranges are disjoint, so a given key sits in at most one file per level.

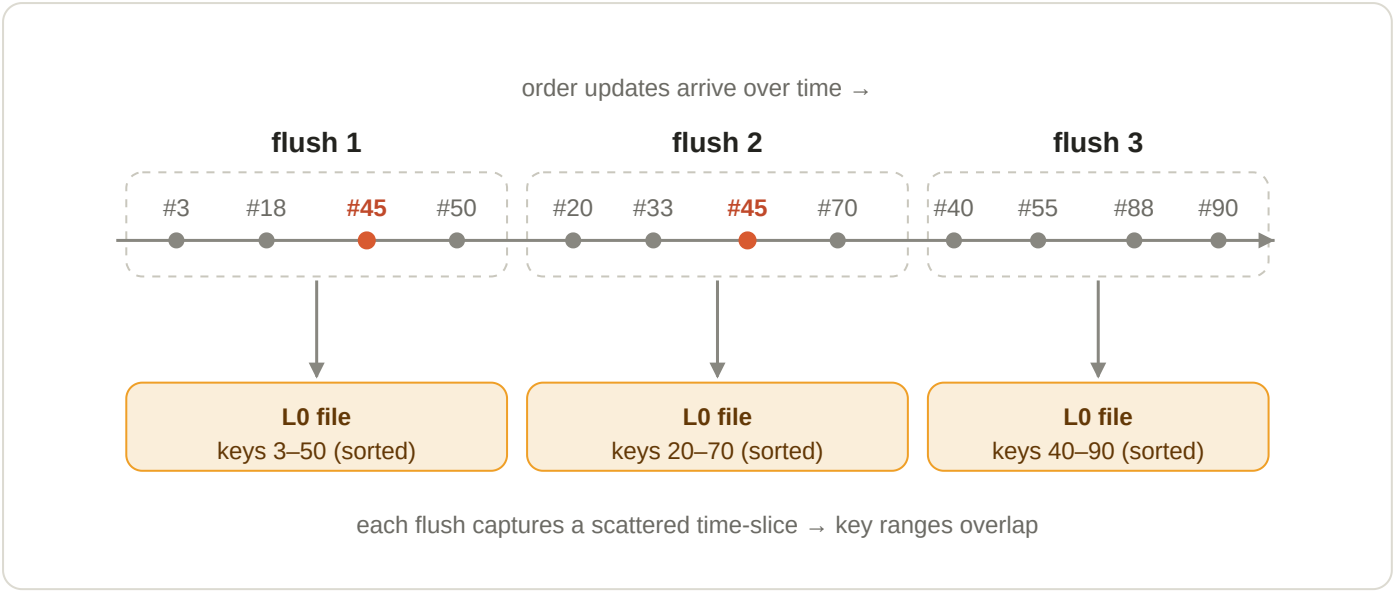
Different sorted runs can still cover the same key with different versions — which is exactly what makes the read path a merge, and the next two sections walk through it.

SECTION 5

Why Level 0 files overlap

The key insight: Paimon flushes files by *time*, not by *key range*.

Each Level 0 file is a snapshot of "whatever keys happened to be written in the last little while." Since any key can be written at any moment — a customer can update order #45 whenever, independently of #3 or #90 — each flush captures a random scatter of keys spanning the whole key space. Two such time-slices therefore overlap as key ranges.



Order #45 is updated in flush 1 and flush 2, so it lands in two L0 files. The ranges 3–50, 20–70, 40–90 overlap.

"Sorted within a file" and "overlapping across files" are not a contradiction. Inside flush 1's file the keys are in order (3, 18, 45, 50); inside flush 2's file they're in order too (20, 33, 45, 70). Each file is internally sorted, but the *interval* [3–50] still overlaps [20–70]. Sorting a batch only orders the keys that are in it; it doesn't restrict which keys are in it.

WHY TOLERATE THE MESS?

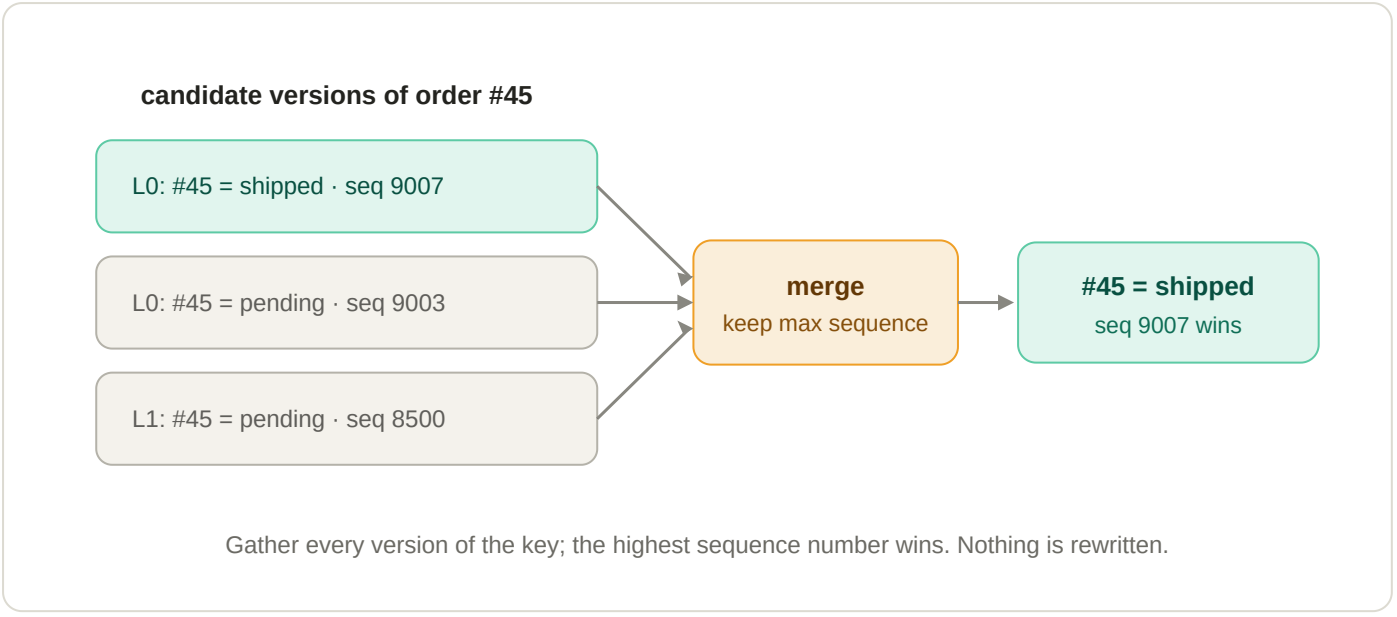
Merging on every flush would *be* copy-on-write again — expensive, on the hot path. Paimon instead accepts a little temporary overlap in Level 0 (cheap writes) and lets background compaction re-partition the key space into the clean, disjoint ranges of Level 1+.

SECTION 6

The read path: merge-on-read in action

Reading a key isn't "open one file" — it's "gather every version and pick the winner."

To return order #45, Paimon checks the memtable and every level that could contain it. Because Level 1+ runs are sorted and non-overlapping, the key sits in at most one file per level there, so most files are skipped; only the overlapping Level 0 needs all-files checking. Every version found is collected, and the one with the highest **sequence number** wins (Section 8). Crucially, this happens entirely in memory at query time — nothing on disk is rewritten.



A point read merges three candidate versions in memory and returns the newest. Scans do the same, key by key.

This merge is the "read tax" you pay for cheap writes — and its size depends directly on how tidy the tree is. If Level 0 has piled up with many overlapping files, every read has to merge more candidates and slows down. That is why the next section's background process, compaction, matters so much: it keeps the read tax small.

WHY PAIMON READS BY PRIMARY KEY ARE FAST ANYWAY

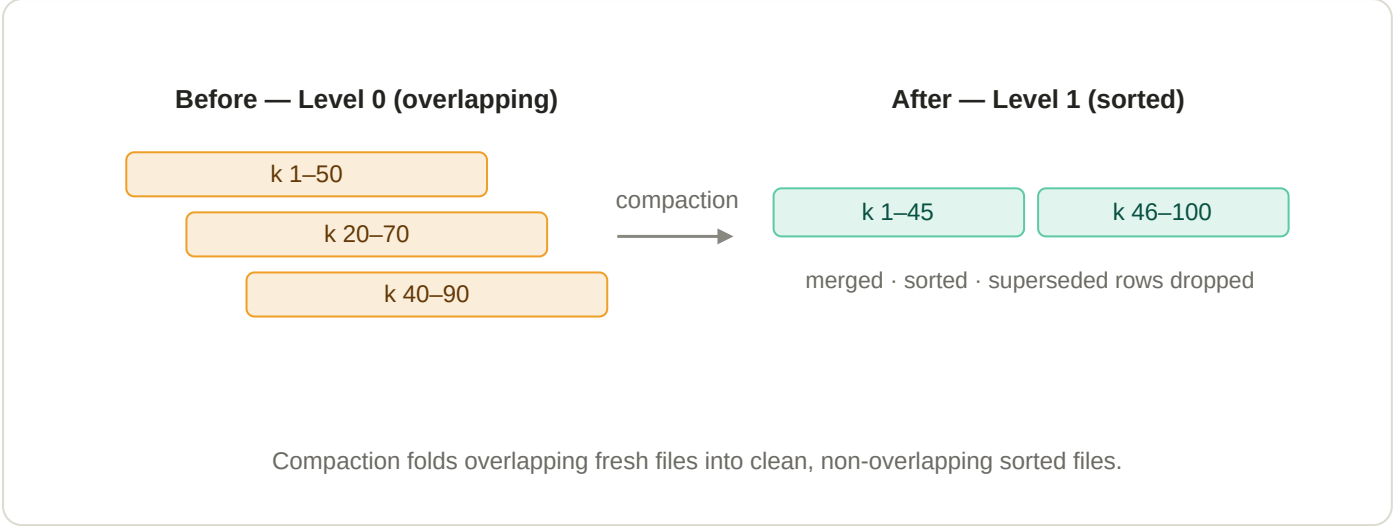
Because data is sorted by key within files and partitioned cleanly across Level 1+, Paimon can use per-file key ranges and indexes to skip almost everything when you filter on the primary key. The merge cost is bounded by how many overlapping runs cover the key, not by table size.

SECTION 7

Compaction & the table modes (MOR / MOW / COW)

Compaction is the background janitor that keeps the read tax from growing without bound.

Compaction takes a pile of small, overlapping files (especially in Level 0), merges them, discards superseded versions, re-sorts by key, and writes the result down into a deeper, tidy level with non-overlapping ranges.



The same key space, re-partitioned. Old superseded versions are physically removed here, not at read time.

The decisive difference from Delta's rewrite-on-every-update is *scheduling*: compaction runs asynchronously, in the background, in batches — not on the critical path of each tiny write. The default trigger is when the number of Level 0 files crosses a threshold (about 5). If compaction falls behind a fast stream, the symptom is predictable: Level 0 file count climbs and reads slow down. That is why "Level 0 files per bucket" is the single most-watched health metric on streaming Paimon tables.

MERGE VS COMPACTION — TWO SEPARATE MECHANISMS

Merge (at read): gather a key's versions, pick the max sequence number, in memory, rewrite nothing. **Compaction (background):** physically merge files and delete the losers on disk, later. Both resolve superseded versions — one transiently per query, one durably.

The three table modes — it's a dial

How aggressively Paimon compacts is configurable, and it sets where you sit on the write-cost / read-cost spectrum.

Paimon's table modes span a single dial. Copy-on-write is just the far end — i.e. Delta's behaviour.

- **Merge-on-read (MOR)** — the default. Only minor compactions; readers merge. Cheapest writes, costlier reads.
- **Merge-on-write (MOW)** — enable deletion vectors (`deletion-vectors.enabled = true`). Paimon marks superseded rows so readers can *filter them out without merging*. A middle ground: writes stay reasonable, reads get

much faster.

- **Copy-on-write (COW)** — force a full merge on every commit (`full-compaction.delta-commits = 1`). Reads are fastest (everything already merged), writes are slowest. This is effectively Delta's model.

THE AHA

Paimon is a **superset**: copy-on-write — Delta's whole strategy — is simply one extreme setting of Paimon's compaction dial. But Paimon can also sit at the cheap-streaming-write end, which Delta fundamentally cannot, because Delta has no LSM tree to absorb the churn.

SECTION 8

Sequence numbers, ordering & RowKind

When versions of a key collide, "newest" is decided by a sequence number — and where that counter lives matters.

Each row carries a **sequence number**: a monotonically increasing counter maintained **per bucket** by that bucket's writer — not per primary key, and not one global counter for the whole table.

WHY PER-BUCKET IS EXACTLY RIGHT

You only ever compare sequence numbers between two rows that share a primary key, and all versions of a key always land in the same bucket. So every comparison you'd ever need happens *inside one bucket*, where its single counter already gives a correct order. A global counter would force every parallel writer to coordinate on one number — destroying the parallelism buckets exist to provide.

Controlling the order

- **Default = input order** — the last record received is the last to merge (and therefore wins).
- `sequence.field` — for streams that can arrive out of order, point Paimon at a column (e.g. `update_time`) so the record with the largest value in that field is treated as latest, regardless of arrival order. Supported types include integer/bigint and timestamps.

RowKind — every row knows what it is

Alongside its sequence number, every row carries a **RowKind** tag:

RowKind	Meaning
`+I`	Insert — a new row.
`-U`	Update before-image — the row's value *before* a change.
`+U`	Update after-image — the row's value *after* a change.
`-D`	Delete — supersedes earlier versions; the key is removed.

A delete is therefore just a `-D` row with a high sequence number. A **superseded version** is any older row for a key that a newer write has replaced — reads ignore it, and compaction physically deletes it. The before/after pair (`-U` / `+U`) becomes essential in the next section, where downstream consumers need both to react correctly.

SECTION 9

Merge engines: how same-key records combine

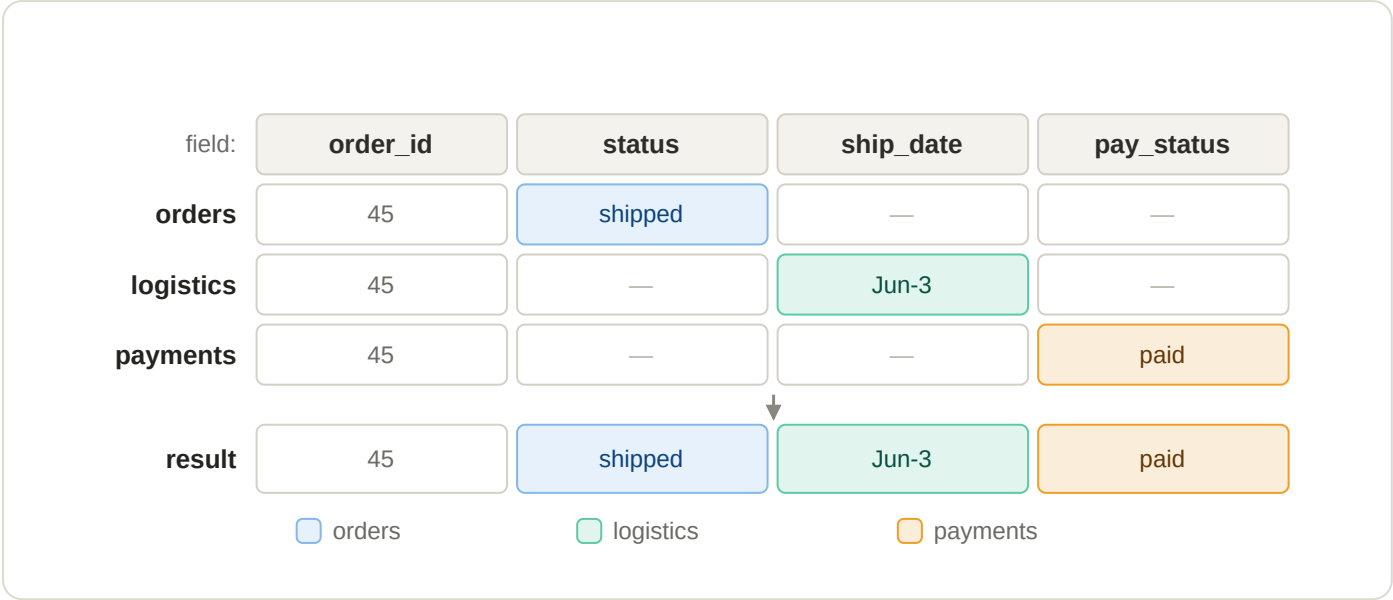
"Newest wins" is only the default. The merge engine — a table property — decides how same-key records combine into one surviving row.

1. deduplicate default

Keep the latest record, throw away the rest. If the latest record is a `-D`, the key is deleted (configurable via `ignore-delete`). This is the "mirror a source table" engine: a CDC stream lands here and the Paimon table equals the source's current state, row for row.

2. partial-update — assemble a wide row from many sources

Each column is filled independently using the latest value, and **null values do not overwrite existing values**. Multiple streams can each contribute their slice of one wide row over time — effectively a "join by primary key, accumulated by the storage layer," with no actual join.



Three streams each fill different columns of order #45; nulls never overwrite, so one complete row is assembled.

PARTIAL-UPDATE GOTCHAS

- **Cannot accept deletes by default.** Choose `ignore-delete`, `partial-update.remove-record-on-delete` (drop the whole row), or sequence-group retraction. A delete from one source is ambiguous — blank that source's columns, drop the whole row, or ignore? — so you must say which.
- **Multi-stream ordering needs sequence-groups.** A single table-wide sequence field can let one stream's stale data overwrite another's; give each source its own ordering column with `fields.<g>.sequence-group`.
- **Streaming reads require a `lookup` or `full-compaction` changelog producer** (Section 10).

3. aggregation — maintain live rollups

Each non-key column is assigned an aggregate function and records fold into a running result — perfect for a summary layer where metrics are maintained by the storage engine instead of a batch job. Columns without an explicit function default to `last_non_null_value`.

```
-- live per-product rollup
CREATE TABLE product_stats (
  product_id BIGINT,
  max_price  DOUBLE,
  total_sales BIGINT,
  PRIMARY KEY (product_id) NOT ENFORCED
) WITH (
  'merge-engine' = 'aggregation',
  'fields.max_price.aggregate-function'  = 'max',
  'fields.total_sales.aggregate-function' = 'sum'
);
-- writing (1, 23.0, 15) then (1, 30.2, 20)  →  result (1, 30.2, 35)
```

Common functions include `sum`, `product`, `count`, `max`, `min`, `first_value` / `first_non_null_value`, `last_value` / `last_non_null_value`, `listagg`, `bool_and` / `bool_or`, `collect`, `merge_map`, plus approximate-distinct sketches (`hll_sketch`, `theta_sketch`).

RETRACTION MATTERS

Only a subset of functions can "undo" an old value when an update/delete arrives — `sum`, `product`, `count`, `collect`, `merge_map`, `last_value`, `last_non_null_value` support retraction; others (e.g. `max`, `min`) cannot un-see a value. For columns that should ignore retraction, set `fields.<name>.ignore-retract = 'true'`. Choose functions that match whether your stream has updates/deletes.

4. first-row — keep the first, ignore the rest

The mirror image of deduplicate: keep the first record seen for each key and ignore all later ones, producing an insert-only changelog. Ideal for de-duplicating noisy append/event streams (e.g. "first event per `event_id`, drop replays"). It doesn't accept deletes, and Level 0 files become visible only after compaction — so its latency profile differs from the others.

Use case	Merge engine
Mirror a source DB table (full CDC, updates + deletes)	`deduplicate`
Assemble a wide entity row from multiple systems	`partial-update`
Maintain live rollups / metrics (sum, count, last)	`aggregation`
De-duplicate a noisy append / event stream	`first-row`

SECTION 10

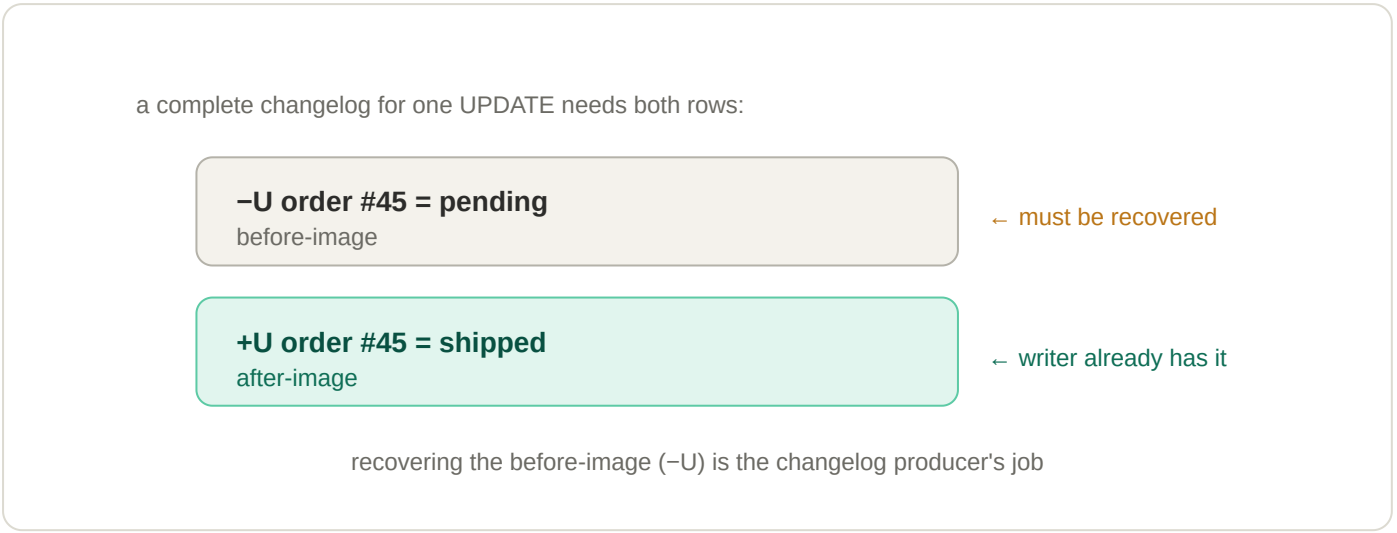
Changelog producers: emitting a correct CDC stream

A primary-key table can emit its own change stream — a binlog of the table's evolution — for downstream consumers.

A **changelog** is a stream of row-level changes. For an update it needs *both* the before-image (`-U`) and after-image (`+U`), because downstream streaming operators — aggregations, joins — must "retract" the old value before applying the new one. If an order total changes 100 → 150, a downstream `SUM` needs to subtract 100 and add 150. Without the before-image, it can't do that correctly.

Why before-images aren't free

This ties straight back to merge-on-read: a streaming writer only holds the *new* row. The old value is buried in an older file the writer never read (MOR writers don't read existing data). So by default Paimon can only expose merged snapshot diffs — not a complete changelog with before-images.



The changelog producer's whole job is to recover the before-image the writer never had.

The four producers

Producer	How it recovers before-images	When to use
`none` **default**	It doesn't — only merged snapshot diffs are exposed.	Batch reads only (batch never needs a changelog).
`input`	Forwards the raw input records as the changelog (a dual write).	The input is already a complete changelog (e.g. database CDC) *and* the engine is `deduplicate`. Cheapest correct option.
`lookup`	Looks up the old value before commit and emits correct −U/+U pairs.	Low-latency complete changelog, or when the input can't be trusted. Resource-hungry.
`full-compaction`	Diffs the results of consecutive full compactions.	Latency of minutes is acceptable; cheaper, decoupled from the write path.

Two sharp edges

- **`input` is a pass-through, not a reconciler.** It is only correct when the table is a `deduplicate` mirror of an already-complete source changelog. The moment the engine transforms data (partial-update, aggregation), the input

no longer equals the table's real change — switch to `lookup`.

- **`full-compaction` is "complete" only at `delta-commits = 1`.** With a larger value, intermediate changes between compactions are collapsed into net changes rather than every change.

> **COST WARNING**
>
> Enabling any changelog producer significantly reduces compaction/write performance. Only turn one on when a downstream *streaming* consumer genuinely needs the change stream — pure batch consumers never do.

SECTION 11

Delta vs Paimon — at a glance

A summary of the contrasts drawn throughout this document.

Dimension	Delta Lake	Apache Paimon
Core data structure	Flat Parquet files + an append-only transaction log	An LSM tree per bucket, tracked by snapshots and manifests
Update strategy	Copy-on-write — pay at write time	Merge-on-read by default; MOW and COW available on the same dial
Streaming upserts	Expensive (heavy write amplification)	Purpose-built — small cheap writes, background compaction
Primary keys	Not part of the model	First-class; data is sharded by key into buckets
Combining same-key records	Single behaviour (latest overwrites)	Pluggable merge engines: deduplicate, partial-update, aggregation, first-row
Read cost	Low and constant (open live files)	A merge whose cost depends on how many overlapping runs cover the key
Native change stream	Change Data Feed	Changelog producers: none / input / lookup / full-compaction
Ordering of versions	By commit order	By per-bucket sequence number (or a chosen `sequence.field`)
Sweet spot	Batch-heavy, append-mostly analytics	High-frequency streaming updates / CDC into the lake

THE ONE-SENTENCE SUMMARY

Paimon swaps Delta's copy-on-write for an LSM tree with merge-on-read, which makes streaming primary-key updates cheap; everything else — buckets, compaction, table modes, merge engines, and the changelog model — exists to manage the consequences of that one swap.

Reference: Apache Paimon documentation (master / 1.x). Prepared as an internal learning aid; verify version-specific option names against the release you deploy.